



Tecnológico de Monterrey

Compresión de telemetría

Franco De Escondrillas Vázquez | A01739410

Octavio Hernández Loyo | A01739304

Ingeniería en Tecnologías Computacionales

Dr. Sandra Eugenia Barajas Montiel

TC2038

5 de octubre, 2026

1. Ejecución del algoritmo

```
Shell
→ A01739304-A01739410_Act3.1 g++ main.cpp Huffman.cpp Mtf.cpp Rle.cpp -o x &&
./x
--- INFORME DE PIPELINE DE COMPRESION ---
Tamano original: 147 bytes (1176 bits)
Paso 1 (MTF Aplicado): 147 elementos generados.
Paso 2 (RLE Aplicado): 131 bloques de corridas compactados.
Paso 3 (Huffman Aplicado): 648 bits totales (81 bytes).
Pipeline de compresion ejecutado exitosamente.
```

Código 1. Ejecución del programa proporcionado por la profesora.

2. Preguntas para el análisis

2.1 ¿Por qué en la función `aplicarMtf` se inicializa un alfabeto de 256 caracteres y cuál es el impacto en tiempo de ejecución de realizar operaciones de `find` y `erase` sobre `std::string` para cadenas largas?

Move to front es sensible al tamaño de su alfabeto, pues cada vez que se encuentra un símbolo en el mensaje, este debe buscarse en la estructura auxiliar, eliminarse y pasarse al frente. Dado que el orden en la lista es dinámico, no se podría implementar como un BST o alguna otra estructura que elimine la búsqueda lineal y permita hacer consultas en $O(\log_2 n)$.

La mejor opción, es usar una lista ligada, en donde buscar tiene una complejidad igual al tamaño del alfabeto $O(\log_2 |\Sigma|)$ (en este caso, 256). La función de eliminar y reinsertar al frente en una lista ligada tienen una complejidad de $O(1)$, por lo que la complejidad total de *Move to Front* es $O(n \cdot |\Sigma|)$.

2.2 En la función `aplicarHuffman`, observa cómo se calcula la variable `claveSimbolo` (`int claveSimbolo = datosRle[i].first * 1000 + datosRle[i].second;`). ¿Qué limitación lógica o de diseño tiene este enfoque si los valores de RLE superan ciertos rangos, y cómo lo rediseñarías utilizando `std::pair` como clave en el mapa?

Lo que hace este algoritmo es calcular un hash sencillo para representar cada pareja. Como el primer término se separa del segundo como

$$hash = p_1x + p_2 \quad (x = 1\,000)$$

entonces los hashes se mantienen únicos siempre y cuando el segundo valor no supere el orden de 10^3 . Dado que el segundo valor es la frecuencia, un archivo de algunos megabytes (como una imagen), podría fácilmente exceder esta cantidad.

Por ello, proponemos cambiar el orden de los términos del hash, y calcularlo como

$$hash = p_2 \cdot |\Sigma| + p_1$$

de esta manera, la frecuencia se multiplica por el tamaño del alfabeto, produciendo números más pequeños y aprovechando mejor la escala numérica. Sabiendo que $0 \leq p_1 < |\Sigma|$, entonces no existe un riesgo de colisión con este método.

2.3 Uso de Min-Heap: Explica por qué se utiliza un objeto de comparación CompararNodos junto con `std::priority_queue` para simular un árbol de Huffman y cómo esto garantiza la propiedad de optimalidad de los códigos prefijo.

En C++, la estructura `std::priority_queue` implementa un heap. El algoritmo de Huffman, inserta los valores como parejas de frecuencia-valor, por lo que el Min-Heap asegura que se tomarán primero los valores que se repiten la menor cantidad de veces. Como el árbol de Huffman se implementa desde las hojas hacia la raíz, los valores menos frecuentes reciben los códigos más largos, mientras que los que más aparecen reciben códigos cortos. De esta manera, se alcanza una condición óptima en donde se repiten muchas veces los códigos cortos, y muy pocas veces los largos.

3. Análisis resultado

¿Tu solución garantiza un proceso de compresión lossless (se recupera exactamente la misma información) al 100%?

Sí, la solución garantiza un proceso de compresión lossless. Esto se puede observar porque, al descomprimir los datos, se recupera el mensaje en su totalidad. Algo que se debe tomar en cuenta es que, para que el proceso pueda llevarse a cabo, se debe incluir en los metadatos del mensaje el alfabeto utilizado, una vez que se termina de aplicar MTF.

```
Shell
→ A01739304-A01739410_Act3.1 g++ main.cpp Huffman.cpp Mtf.cpp Rle.cpp -o x &&
./x
--- INFORME DE PIPELINE DE COMPRESION ---
Tamano original: 147 bytes (1176 bits)
Paso 1 (MTF Aplicado): 147 elementos generados.
Paso 2 (RLE Aplicado): 131 bloques de corridas compactados.
Paso 3 (Huffman Aplicado): 648 bits totales (81 bytes).
Pipeline de compresion ejecutado exitosamente.

--- INFORME DE PIPELINE DE DESCOMPRESION ---
Paso 1 (Invertir Huffman): 131 / bloques recuperados
Paso 2 (Invertir RLE): 147 / elementos recuperados
Paso 3 (Invertir MTF): Se recupero un mensaje de 147 bytes
Se recuperaron 147 / 147bytes
El mensaje decodificado es identico

Mensaje original:
STATUS:OK;TEMP:23.4;PRES:1013;STATUS:OK;
TEMP:23.4;PRES:1013;STATUS:OK;TEMP:23.5;
PRES:1012;SYS:STABLE;SYS:STABLE;SYS:STABLE;
ERR:NONE;ERR:NONE;

Mensaje decodificado:
STATUS:OK;TEMP:23.4;PRES:1013;STATUS:OK;
TEMP:23.4;PRES:1013;STATUS:OK;TEMP:23.5;
PRES:1012;SYS:STABLE;SYS:STABLE;SYS:STABLE;
ERR:NONE;ERR:NONE;
```

Código 2. Ejecución del programa después de implementar los inversos a cada método para la decodificación.